

Case Study: 2FA Authentication

Session Hijacking • Stolen-Password Account Verification • Forgotten 2FA PIN Recovery

Background:

What is 2FA?

Two-factor authentication is a security method that adds a second layer of identity verification. Instead of relying on just a password, 2FA requires you to confirm your identity using two distinct factors. This reduces the risk of unauthorized access, even if a password is compromised.

How 2FA work?

The factors that make up 2FA include:

- **Something you know:** a password, PIN, or passphrase.
- **Something you have:** a physical device like a smartphone, security token, passkey, or smart card.
- **Something you are:** biometric identifiers such as a fingerprint, facial recognition, or voice match.

When a system uses exactly two of these categories, it qualifies as two-factor authentication. This makes 2FA a specific subset of multifactor authentication(MFA), which may use two or more factors.

Source: [www.microsoft.com](https://www.microsoft.com/en-us/security/devices-accounts/multi-factor-authentication)

Introduction

This case study talks about the 2 problems/loopholes in 2FA/MFA. They are:

(1) Session Hijacking and

(2) recovery when a user forgets a 2FA PIN. The three problems are treated as parts of the same broader security challenge: protecting authenticated identities while preserving usable, recoverable access.

Problem 1 — Session Hijacking

Session hijacking occurs when an attacker obtains a valid session identifier after a legitimate user has authenticated and then uses that token to impersonate the user. The attackers establish a session through a unique session ID stored in cookies or URLs, followed by compromise through:

Session sniffing: Capturing unencrypted network traffic (e.g., via tools like Wireshark) on public Wi-Fi.

Cross-site scripting (XSS): Injecting malicious scripts into websites to exfiltrate cookies.

Session fixation: Forcing users to adopt a predetermined session ID via phishing links.

Infostealer malware: Harvesting browser cookies and fingerprints from infected devices. Once stolen, the token can provide access without requiring the attacker to repeat the original login process.

Proposed Solution — Session Hijacking

The developers propose a layered defense. First, enforce encryption with HTTPS/TLS, HSTS, and Secure/HttpOnly cookie controls.

Second, strengthen session management through session-ID regeneration after login or privilege changes, short inactivity timeouts, and token/device binding.

Third, use MFA for sensitive systems and high-risk actions. Although they are not fully invincible, they do give a certain friction to the attack.

Fourth, train users to avoid unsafe networks, recognize phishing, and log out of shared devices.

Fifth, continuously monitor geographic impossibilities, device mismatches, and concurrent sessions and prepare proactive incident response through automated session termination, real-time alerts, password resets, and token revocation.

Session-Hijacking Security Flow

The security flow below shows how a legitimate session can become compromised and where the proposed controls intervene.

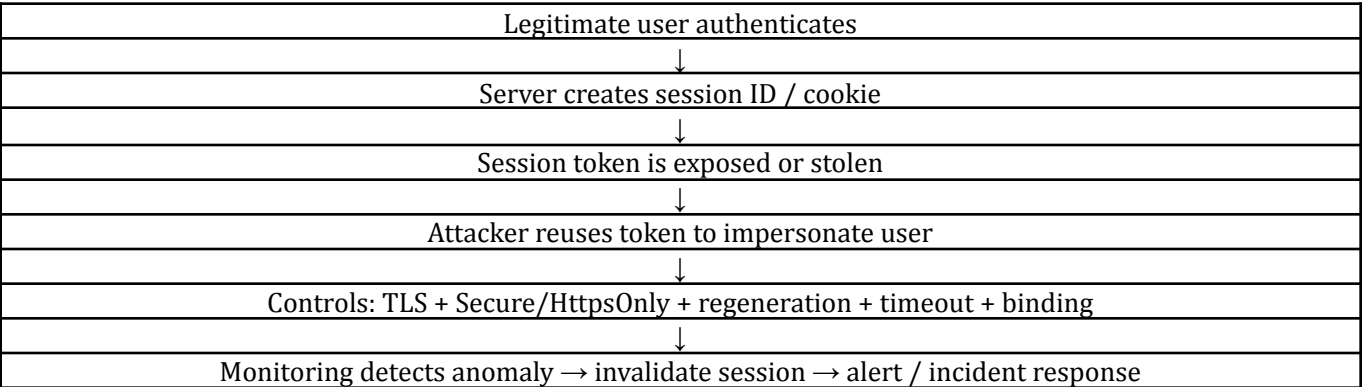


Figure 1. Session-hijacking attack path and defensive controls.

Problem 2 — User Forgets 2FA PIN

When a user forgets a 2FA PIN, the security mechanism can become a barrier to legitimate access. This creates a trust problem: the system must allow the legitimate owner to recover the account without creating an easier path for an attacker. The recovery mechanism therefore has to use an independently verifiable channel already associated with the account.

Proposed Solution — Forgot PIN Recovery

The proposed solution is a “Forgot PIN” flow using a one-time, time-limited OTP sent to the user's verified phone number or email. The PIN is hashed rather than stored in a recoverable form, so recovery means resetting the PIN rather than retrieving it.

Implementing rate limits restrict PIN attempts and OTP requests.

Backup codes provide another fallback when phone/email access is unavailable. After a successful reset, all other active sessions/devices are invalidated and a reset notification is sent to the verified contact.

And finally audit logging records recovery activity.

Forgot-PIN System Architecture

Our study specifies a React frontend, Django/Django REST Framework backend, OTP service, rate-limiting middleware, database-backed PIN hashing and reset logic, audit logging, and email/SMS notification dispatch. PostgreSQL stores users, OTP requests, backup codes, and audit records.

Redis supports short-lived OTP data and rate-limit counters.

SMS/email gateways deliver OTPs and reset alerts.

The flow is: React recovery UI → Django REST API → OTP service/database verification → PIN-reset token → new PIN screen → hashed PIN update → invalidation of other sessions.

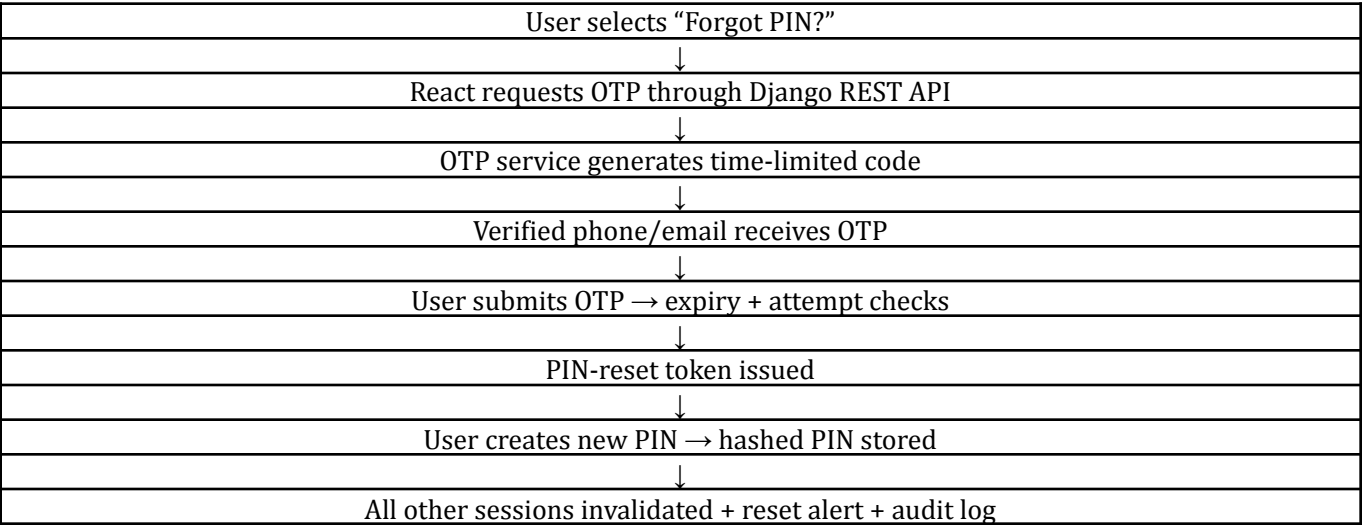


Figure 2. Forgot-PIN recovery flow.

Technology Used and Why

The technologies are selected according to the specific security and usability requirements.

Problem / Area	Technology / Control	Why Used
Session hijacking	HTTPS/TLS, HSTS, Secure/HttpOnly cookies	Protect traffic and restrict unsafe cookie access.
Session hijacking	Session regeneration, timeouts, token binding	Reduce replay and exposure of active session tokens.

Session hijacking	Monitoring + incident response	Detects anomalies and terminates compromised sessions.
Forgot PIN	React	Componentized multi-step recovery UI.
Forgot PIN	Django + DRF	Secure REST endpoints and authentication/recovery services.
Forgot PIN	PostgreSQL	Relational integrity for users, OTPs, backup codes, and audit logs.
Forgot PIN	Redis	TTL-based OTP expiry and rate-limit counters.
Forgot PIN	Twilio / SendGrid	Production-grade SMS/email delivery.
Forgot PIN	bcrypt / Argon2	Slow, purpose-built credential hashing.
Forgot PIN	JWT / server-side sessions	Supports centralized session invalidation after reset.

Features Implemented

Across the two problem areas, the study describes security controls and user-facing features including encrypted web traffic, secure cookie attributes, session-ID regeneration, short timeouts, token/device binding, MFA for sensitive actions, anomaly monitoring, automated session termination, Forgot PIN recovery, OTP expiry and resend controls, rate limiting, PIN-strength rules, full session invalidation after reset, reset notifications, and audit logging.

Integrated End-to-End Security Flow

The two solutions can be viewed as complementary layers. Session security protects the authenticated session after login and the Forgot PIN mechanism protects the recovery boundary when the legitimate user cannot remember the configured 2FA PIN.

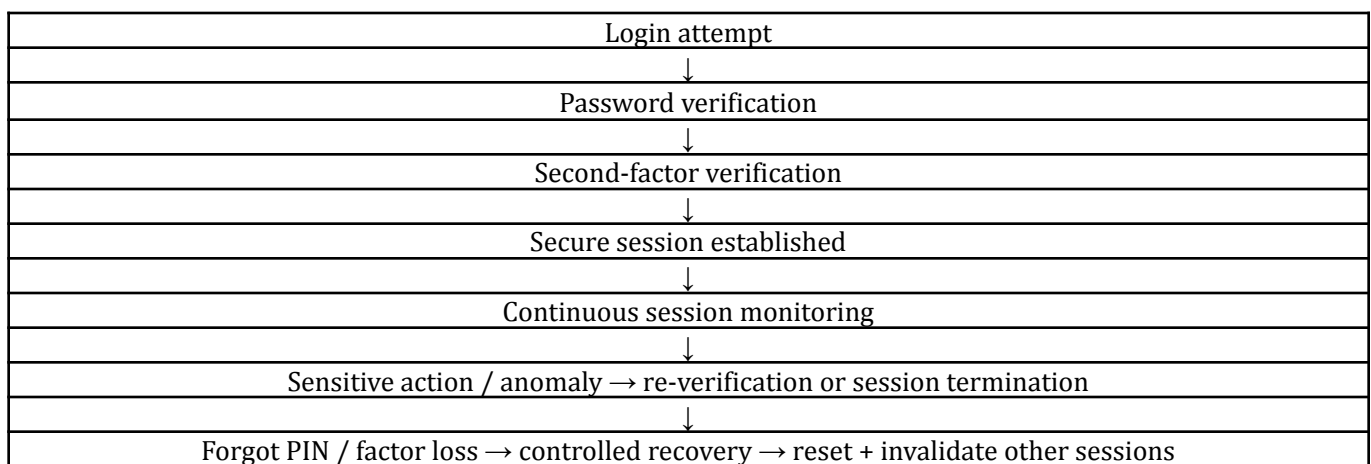


Figure 4. Integrated lifecycle view of authentication, session protection, and recovery.

Overall Summary — Solving the Three Problems

The source demonstrates that authentication security is not solved by adding one extra login field. The two cases require a lifecycle approach: secure the session,, and design recovery with the same security mindset. For session hijacking, encryption, stronger session controls, monitoring, and incident response reduce token theft and misuse. For forgotten 2FA PINs, OTP-based recovery, backup codes, rate limits, session invalidation, notifications, and audit logs provide a self-service recovery path without creating a hidden backdoor.

Team Learning

The study shows several engineering lessons: security features introduce new services and failure modes; open standards such as TOTP and WebAuthn avoid reinventing cryptographic trust; recovery must be designed alongside authentication; usability is part of security because excessive friction can cause abandonment or unsafe behavior; and large-scale security changes require careful rollout, communication, and support. Diagramming the architecture before implementation also makes trade-offs and failure points visible early.

Source/References:

- www.microsoft.com
- www.obsidiansecurity.com
- www.proofpoint.com
- www.pages.nist.gov
- www.cheatsheetseries.owasp.org
- www.en.wikipedia.org
-